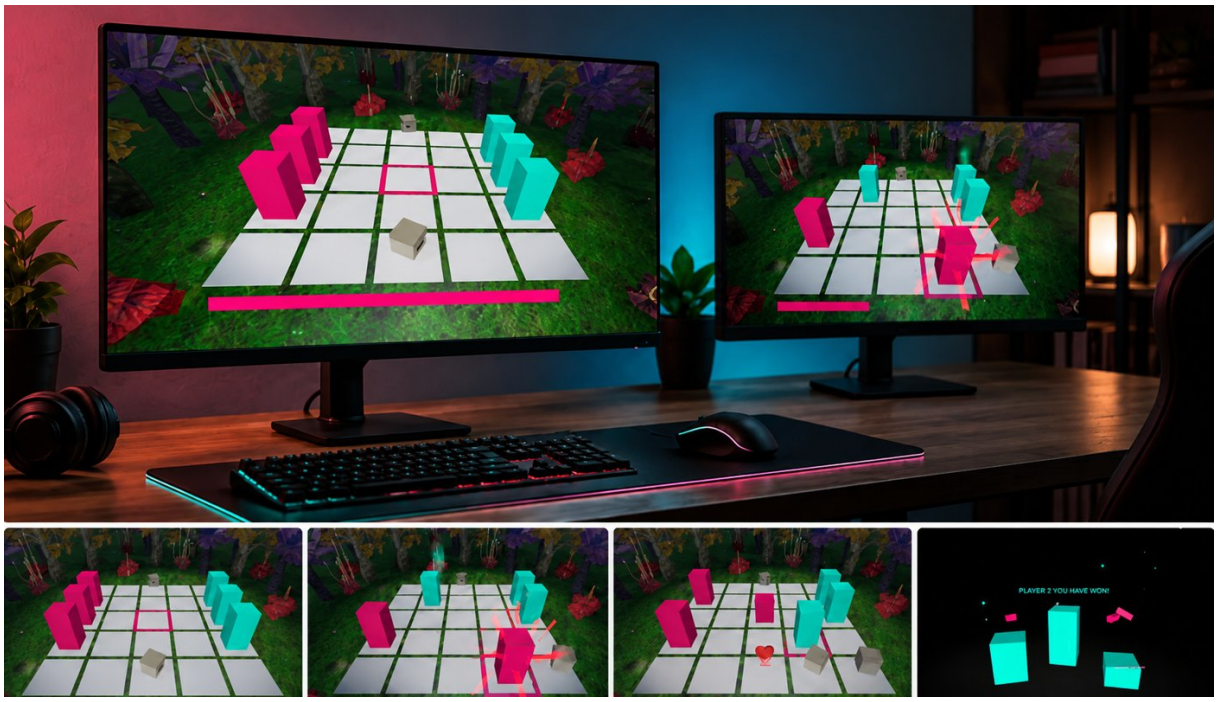


Real-Time Tactics

Estrategia táctica 1 contra 1 en Unity

Eva Gallardo Romero

29 de abril de 2025

Índice

Enlace al repositorio de GitHub **2**

1 Descripción general **2**

2 Diagrama de clases **2**

3 Diseño **3**
3.1 Controles 3

4 Requisitos del sistema **4**
4.1 Requisitos funcionales 4
4.2 Requisitos no funcionales 4

5 Documentación técnica **5**
5.1 GameManager.cs 5
5.2 Board.cs 7
5.3 CursorLogic.cs 8
5.4 Piece.cs 9
5.5 Crate.cs y HealthPickup.cs 10

Enlace al repositorio de GitHub

Puedes consultar el proyecto completo en el siguiente enlace: Real-Time-Tactics.

El juego se puede probar en el navegador en itch.io y la documentación del código generada con Doxygen está en evagr11.github.io/Real-Time-Tactics.

1 Descripción general

Real-Time Tactics es un juego de **estrategia táctica para dos jugadores** en el mismo teclado, hecho en **Unity 3D** con **C#** en la asignatura de Programación de 1.º de DAM. Se juega en un **tablero de 5×5** ambientado en un bosque de noche.

- Cada jugador tiene **3 piezas con 3 puntos de vida**. La altura de cada pieza muestra su vida: se encoge al recibir daño.
- Los turnos son **en tiempo real**: cada jugador tiene **15 segundos** para hacer todo lo que pueda. Una barra de su color marca el tiempo que queda.
- Con el cursor se **agarra** una pieza propia, se mueve casilla a casilla y se suelta. Al **atacar**, la pieza golpea las cuatro casillas de alrededor y quita 1 punto de vida a cada rival.
- Después de atacar, la pieza tiene un **tiempo de espera**. Ese tiempo es más corto cuantas menos piezas le quedan al jugador, para dar opciones de **remontada**.
- Cada 5–15 segundos aparecen **cajas** (hasta 3). Al romperlas dejan un **corazón** que cura 1 punto a la pieza que lo coge.
- Gana quien elimine todas las piezas del rival. Un panel baja y se carga la escena del ganador.

El cursor es **toroidal**: si sale por un borde del tablero entra por el contrario.

2 Diagrama de clases

El código está separado en **modelo** (tablero, piezas, cajas y corazones), **controlador** (`GameManager` y `CursorLogic`) y **vista** (los scripts que animan cada objeto). Todo lo que puede estar en una casilla implementa `IGameEntity`, y lo que se puede atacar, `IAttackable`.

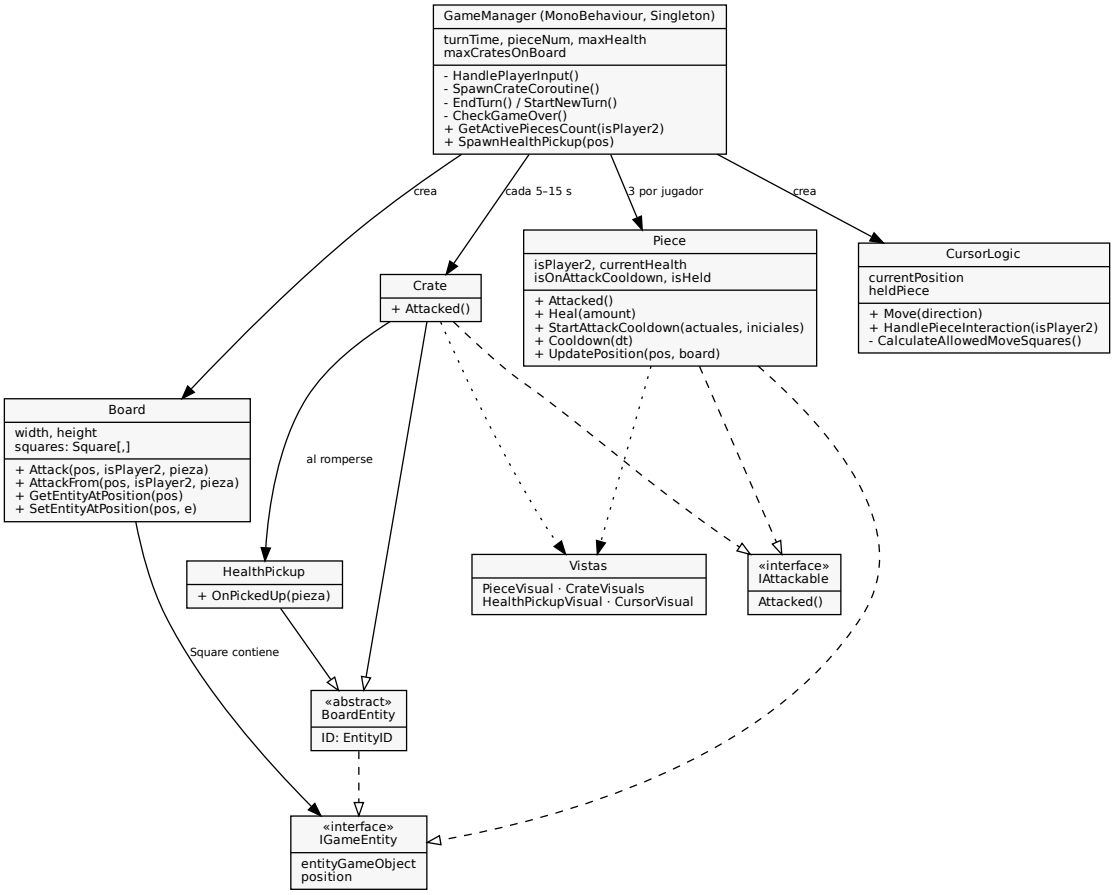


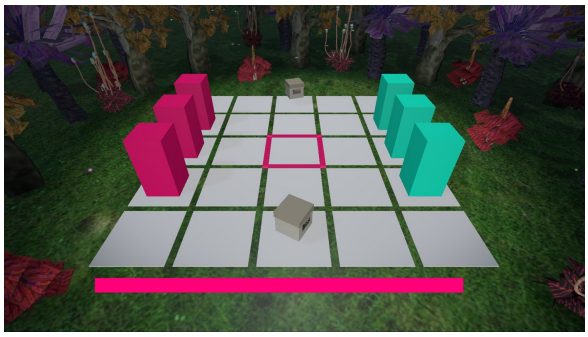
Figura 1: Diagrama de clases de Real-Time Tactics

3 Diseño

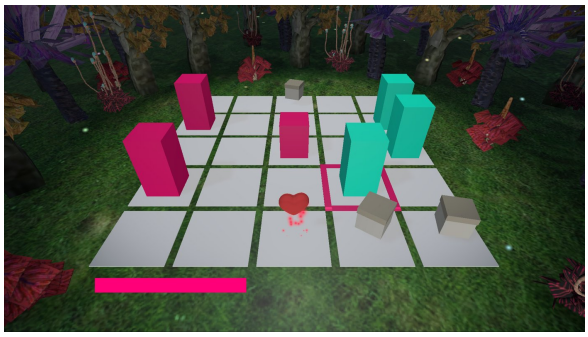
El tablero son casillas blancas sobre un claro del bosque, con árboles, plantas y luciérnagas alrededor. Las piezas del jugador 1 son rosas y las del 2, turquesa; el cursor y la barra de tiempo toman el color del jugador que tiene el turno. Las cajas son grises y el corazón, rojo.

3.1 Controles

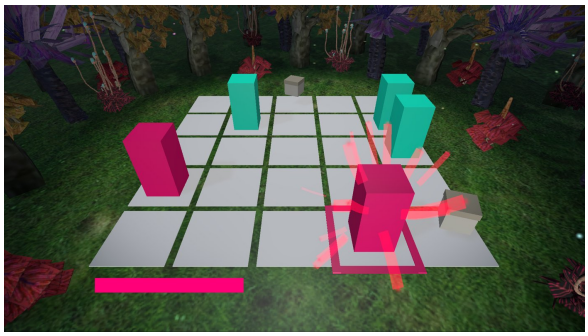
Acción	**Jugador 1**	**Jugador 2**
Mover el cursor	W A S D	Flechas
Agarrar / soltar	Ctrl izquierdo	Ctrl derecho
Atacar	Espacio	Enter



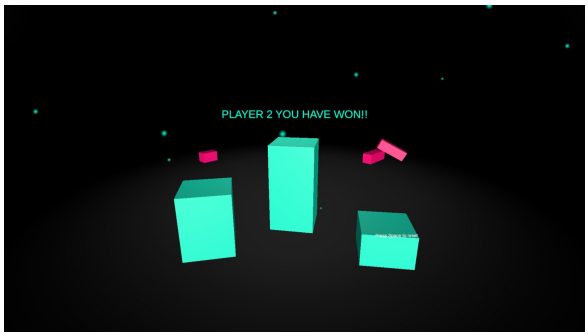
Inicio de la partida



Caja rota y corazón



Pieza con partículas



Escena del ganador

Figura 2: Capturas de una partida

4 Requisitos del sistema

4.1 Requisitos funcionales

ID	Requisito funcional
RF01	El juego debe crear un tablero de **5×5** con 3 piezas por jugador en filas opuestas.
RF02	El primer turno se debe sortear y cada turno debe durar **15 segundos**.
RF03	El cursor debe moverse casilla a casilla y **dar la vuelta** por los bordes.
RF04	Un jugador solo puede agarrar **sus piezas**, y moverlas a casillas vacías o con un corazón.
RF05	Al atacar, la pieza agarrada debe golpear las **4 casillas adyacentes**: quita 1 de vida a las piezas rivales y rompe las cajas.
RF06	Después de atacar, la pieza no puede volver a hacerlo hasta que pase su tiempo de espera (2 s × piezas que quedan / piezas iniciales, como mínimo el 20 %).
RF07	La altura de la pieza debe ser proporcional a su vida.
RF08	Deben aparecer cajas en casillas libres cada 5–15 segundos, con un máximo de 3.
RF09	Al romper una caja debe aparecer un corazón que cura 1 punto, sin pasar de la vida máxima.
RF10	Cuando un jugador se queda sin piezas, debe cargarse la escena de victoria del otro.

4.2 Requisitos no funcionales

ID	Requisito no funcional
RNF01	El juego debe estar hecho en **Unity 3D** con scripts en **C#**.
RNF02	La lógica (modelo y controlador) debe estar separada de los scripts visuales.
RNF03	Los valores del juego (tamaño, tiempo de turno, vida, cajas) deben poder cambiarse desde el Inspector.
RNF04	Dos personas deben poder jugar en el **mismo teclado**.
RNF05	Debe poder jugarse en el navegador (versión WebGL en itch.io).
RNF06	Los scripts deben tener comentarios de documentación `///` para Doxygen.

5 Documentación técnica

Este anexo agrupa la documentación por scripts, separándola en la **ubicación** donde se encuentra, una **descripción general** y las **funciones** explicadas de manera individual con su **código** adjunto y comentado. Se omiten los `using`, los comentarios `///` y los scripts de la vista, que solo animan los objetos.

5.1 GameManager.cs

5.1.1 Ubicación

`Assets/Scripts/Controller/GameManager.cs`

Descripción general:
Controla la partida: crea el tablero, el cursor y las piezas, lee el teclado del jugador que tiene el turno, cuenta el tiempo, genera cajas y comprueba si alguien ha ganado. Es un *singleton* (`GameManager.Instance`).

5.1.2 Funciones

Start **Descripción:** Prepara la partida, sortea quién empieza y arranca la corrutina de las cajas.

Código comentado:

```
void Start()
{
    board = new Board(boardWidth, boardHeight, boardSquarePrefab);
    Vector2Int cursorStartPos = new Vector2Int(boardWidth / 2, boardHeight / 2);
    cursorLogic = new CursorLogic(boardWidth, boardHeight, cursorStartPos, board);
    // ... se crea el cursor visual en el centro
    InitializePieces();

    if (startWithRandomPlayer)
        isPlayer2Turn = (Random.Range(0, 2) == 0);      // 50 %
    else
        isPlayer2Turn = false;

    StartNewTurn();
    StartCoroutine(SpawnCrateCoroutine());
}
```

HandlePlayerInput **Descripción:** Lee solo las teclas del jugador del turno y se las pasa al cursor o al tablero. Las flechas se giran porque la cámara mira el tablero de lado.

Código comentado:

```
if (isPlayer2Turn)
{
    if (Input.GetKeyDown(KeyCode.UpArrow)) moveDirection = Vector2Int.left;
    else if (Input.GetKeyDown(KeyCode.DownArrow)) moveDirection = Vector2Int.right;
    else if (Input.GetKeyDown(KeyCode.LeftArrow)) moveDirection = Vector2Int.down;
    else if (Input.GetKeyDown(KeyCode.RightArrow)) moveDirection = Vector2Int.up;
```

```
    if (Input.GetKeyDown(KeyCode.RightControl)) interactPressed = true;
    if (Input.GetKeyDown(KeyCode.Return)) attackPressed = true;
}
else
{
    // mismo esquema con W A S D, Ctrl izquierdo y Espacio
}

if (moveDirection != Vector2Int.zero)
    cursorLogic.Move(moveDirection);
if (interactPressed)
    cursorLogic.HandlePieceInteraction(isPlayer2Turn);
if (attackPressed)
    board.AttackFrom(cursorLogic.currentPosition, isPlayer2Turn, GetCurrentPlayerPiece
        ());
```

FixedUpdate y EndTurn **Descripción:** Descuenta el tiempo del turno. Al acabar, suelta la pieza que estuviera agarrada y pasa el turno.

Código comentado:

```
void FixedUpdate()
{
    if (timeLeftInTurn > 0)
    {
        timeLeftInTurn -= Time.fixedDeltaTime;
        if (timeLeftInTurn <= 0)
        {
            timeLeftInTurn = 0;
            EndTurn();
        }
    }
}

void EndTurn()
{
    cursorLogic.HandlePieceInteraction(isPlayer2Turn);   // suelta la pieza si la hay
    isPlayer2Turn = !isPlayer2Turn;
    StartNewTurn();                                      // reinicia el tiempo y los
        colores
}
```

SpawnCrateCoroutine y SpawnCrate **Descripción:** Mientras haya menos cajas que el máximo, pone una en una casilla libre al azar y espera entre 5 y 15 segundos.

Código comentado:

```
IEnumerator SpawnCrateCoroutine()
{
    while (true)
    {
        if (CountCratesOnBoard() < maxCratesOnBoard)
            SpawnCrate();
        yield return new WaitForSeconds(Random.Range(minCrateSpawnCD, maxCrateSpawnCD)
            );
    }
}

void SpawnCrate()
{
    Vector2Int position;
    do
    {
        position = new Vector2Int(Random.Range(0, boardWidth), Random.Range(0,
            boardHeight));
    } while (board.GetSquareAtPosition(position.x, position.y).containedEntity != null
              || position == cursorLogic.currentPosition);       // libre y sin el
                  cursor
    Crate crate = new Crate(position, EntityID.Crate, cratePrefab);
    board.GetSquareAtPosition(position.x, position.y).containedEntity = crate;
}
```

CheckGameOver **Descripción:** Si a un jugador no le quedan piezas, baja el panel y carga la escena de victoria del otro.

Código comentado:

```
void CheckGameOver()
{
    bool player1HasPieces = allPieces.Exists(piece => !piece.isPlayer2);
    bool player2HasPieces = allPieces.Exists(piece => piece.isPlayer2);
    if (!player1HasPieces)
        StartCoroutine(LowerPanelCoroutine(panel, targetY, speed, "Win2"));
    else if (!player2HasPieces)
        StartCoroutine(LowerPanelCoroutine(panel, targetY, speed, "Win1"));
}
```

5.2 Board.cs

5.2.1 Ubicación

`Assets/Scripts/Model/Board.cs`

Descripción general:
El tablero lógico: una matriz de casillas (`Square`) que guardan la entidad que hay en cada una.

5.2.2 Funciones

Attack **Descripción:** Ataca una casilla. Si hay una pieza rival y la pieza atacante no está en espera, le quita vida y empieza la espera; si hay una caja, la rompe.

Código comentado:

```
public bool Attack(Vector2Int position, bool isPlayer2Attacker, Piece attackingPiece)
{
    if (IsOutOfBounds(position) || !attackingPiece.isHeld)
        return false;                       // hay que tener la pieza agarrada

    IGameEntity entity = GetEntityAtPosition(position);
    if (entity == null)
        return false;

    if (entity is Piece targetPiece)
    {
        if (attackingPiece.isOnAttackCooldown)
            return false;
        if (targetPiece.isPlayer2 != isPlayer2Attacker &&
            targetPiece.entityGameObject.activeInHierarchy)
        {
            targetPiece.Attacked();
            int currentPieces = GameManager.Instance.GetActivePiecesCount(
                attackingPiece.isPlayer2);
            int initialPieces = GameManager.Instance.pieceNum;
            attackingPiece.StartAttackCooldown(currentPieces, initialPieces);
            return true;
        }
    }
    else if (entity is IAttackable attackable)   // por ejemplo, una caja
    {
        attackable.Attacked();
        return true;
    }
    return false;
}
```

AttackFrom **Descripción:** Lanza el ataque a las cuatro casillas de alrededor.

Código comentado:

```
public void AttackFrom(Vector2Int position, bool isPlayer2Attacker, Piece
    attackingPiece)
{
```

```
    Vector2Int[] directions = { Vector2Int.up, Vector2Int.down, Vector2Int.left,
        Vector2Int.right };
    foreach (var dir in directions)
        Attack(position + dir, isPlayer2Attacker, attackingPiece);
}
```

5.3 CursorLogic.cs

5.3.1 Ubicación

`Assets/Scripts/Controller/CursorLogic.cs`

Descripción general:
La posición del cursor y la pieza que lleva agarrada.

5.3.2 Funciones

Move **Descripción:** Sin pieza, el cursor da la vuelta por los bordes. Con una pieza agarrada, solo se mueve a casillas permitidas y la pieza va con él.

Código comentado:

```
public void Move(Vector2Int direction)
{
    Vector2Int newPotentialPosition = currentPosition + direction;
    if (!IsHoldingPiece())
    {
        // tablero toroidal para el cursor
        if (newPotentialPosition.x < 0) newPotentialPosition.x = boardWidth - 1;
        else if (newPotentialPosition.x >= boardWidth) newPotentialPosition.x = 0;
        if (newPotentialPosition.y < 0) newPotentialPosition.y = boardHeight - 1;
        else if (newPotentialPosition.y >= boardHeight) newPotentialPosition.y = 0;
        currentPosition = newPotentialPosition;
    }
    else
    {
        CalculateAllowedMoveSquares();
        if (allowedMoveSquares.Contains(newPotentialPosition))
        {
            IGameEntity entityAtTarget = boardReference.GetEntityAtPosition(
                newPotentialPosition);
            if (entityAtTarget == null || entityAtTarget is HealthPickup ||
                newPotentialPosition == originalPickUpPosition)
            {
                currentPosition = newPotentialPosition;
                heldPiece.UpdatePosition(currentPosition, boardReference);
            }
        }
    }
}
```

HandlePieceInteraction **Descripción:** Agarra una pieza propia o suelta la que lleva. Si la suelta sobre un corazón, la pieza se cura.

Código comentado:

```
if (heldPiece == null)
{
    IGameEntity entityOnSquare = boardReference.GetEntityAtPosition(currentPosition);
    if (entityOnSquare is Piece pieceToPick && pieceToPick.isPlayer2 ==
        isCurrentPlayer2)
    {
        heldPiece = pieceToPick;
        heldPiece.isHeld = true;
        originalPickUpPosition = currentPosition;
        boardReference.SetEntityAtPosition(originalPickUpPosition, null);
        CalculateAllowedMoveSquares();
    }
}
```

```
else
{
    IGameEntity entityAtTarget = boardReference.GetEntityAtPosition(currentPosition);
    if (entityAtTarget == null || entityAtTarget is HealthPickup || currentPosition ==
         originalPickUpPosition)
    {
        if (entityAtTarget is HealthPickup healthPickup)
            healthPickup.OnPickedUp(heldPiece);
        boardReference.SetEntityAtPosition(currentPosition, heldPiece);
        heldPiece.UpdatePosition(currentPosition, boardReference);
        heldPiece.isHeld = false;
        heldPiece = null;
        allowedMoveSquares.Clear();
    }
}
```

5.4 Piece.cs

5.4.1 Ubicación

`Assets/Scripts/Model/Piece.cs`

Descripción general:
Una pieza: de qué jugador es, su vida y su tiempo de espera entre ataques.

5.4.2 Funciones

StartAttackCooldown **Descripción:** Empieza la espera. Se multiplica por la proporción de piezas que quedan (mínimo 0,2), así que con menos piezas se ataca más rápido.

Código comentado:

```
public void StartAttackCooldown(int currentPieces, int initialPieces)
{
    if (!isOnAttackCooldown)
    {
        PieceVisual visual = pieceGameObject.GetComponent<PieceVisual>();
        isOnAttackCooldown = true;
        float multiplier = (float)currentPieces / initialPieces;
        if (multiplier < 0.2f)
            multiplier = 0.2f;
        currentAttackCooldownTimer = attackCooldownDuration * multiplier;   // 2 s de
            base
        visual.UpdateCooldownVisual(true, currentAttackCooldownTimer);
    }
}
```

Attacked y Heal **Descripción:** Al recibir un golpe pierde 1 de vida y se encoge; si llega a 0 se destruye. Al curarse no pasa de la vida máxima.

Código comentado:

```
public void Attacked()
{
    currentHealth--;
    PieceVisual visual = pieceGameObject.GetComponent<PieceVisual>();
    visual.ShrinkOnHit(currentHealth, maxHealth);       // altura = vida / vida máxima
    if (currentHealth <= 0)
    {
        visual.PlayDestructionParticles();
        // ... aquí calcula un tiempo de espera nuevo que no llega a usarse
        GameObject.Destroy(pieceGameObject);
    }
}

public void Heal(int amount)
{
    int calculatedHealth = currentHealth + amount;
    if (calculatedHealth > maxHealth)
```

```
18          calculatedHealth = maxHealth;
19      currentHealth = calculatedHealth;
20      pieceGameObject.GetComponent<PieceVisual>().ShrinkOnHit(currentHealth, maxHealth);
21  }
```

5.5 Crate.cs y HealthPickup.cs

5.5.1 Ubicación

`Assets/Scripts/Model`

Descripción general:
La caja se rompe de un golpe y deja un corazón; el corazón cura a la pieza que lo coge.

5.5.2 Funciones

Crate.Attacked **Descripción:** Rompe la caja, avisa con un evento y crea un corazón en su casilla.

Código comentado:

```
1  public void Attacked()
2  {
3      visuals.PlayDestructionParticles();
4      GameObject.Destroy(visuals.gameObject);
5      GameManager.Instance.RemoveCrateAtPosition(position);
6      GameEvents.CrateBroke.Invoke(position);
7      GameManager.Instance.SpawnHealthPickup(position);
8  }
```

HealthPickup.OnPickedUp **Descripción:** Cura 1 punto si a la pieza le falta vida y desaparece.

Código comentado:

```
1  public void OnPickedUp(Piece piece)
2  {
3      if (piece.currentHealth < piece.maxHealth)
4      {
5          piece.Heal(1);
6          piece.pieceGameObject.GetComponent<PieceVisual>().PlayHealParticles();
7      }
8      GameManager.Instance.RemoveHealthPickupAtPosition(position);
9      GameObject.Destroy(visual.gameObject);
10 }
```